Extending CAESURA: Robustness and Automated Trajectory Logging for SLM Fine-Tuning

AIDM Seminar 2025/2026 Report

Daril Vergesse Sobze Soffack
Technical University of Darmstadt
Darmstadt, Germany

1 MOTIVATION & OBJECTIVE

As modern data systems increasingly store non-relational, multi-modal data (e.g., text, images), extracting insights requires complex data processing pipelines. The CAESURA framework introduced Language-Model-Driven Query Planning to automatically translate natural language into executable multi-modal query plans [1].

Translating these queries requires non-trivial reasoning over the user's intents and the available multi-modal operators. While massive frontier models (e.g., GPT-4) can perform this zero-shot reasoning, relying on them for every database transaction introduces severe latency and cost overheads at query runtime. A highly attractive alternative is to fine-tune Small Language Models to act as specialized query planners. However, this approach faces a critical practical bottleneck: fine-tuning foundation models requires substantial domain-specific data collection. Manually annotating complex, multi-modal logical query plans is prohibitively expensive, tedious, and scales poorly.

As illustrated in Figure 1, the primary objective of this work was two-fold: (1) to solve this data generation bottleneck by transforming the environment into an automated data engine, and (2) to modernize the underlying CAESURA infrastructure so it can reliably support high-throughput experimentation with modern, open-source LLMs.

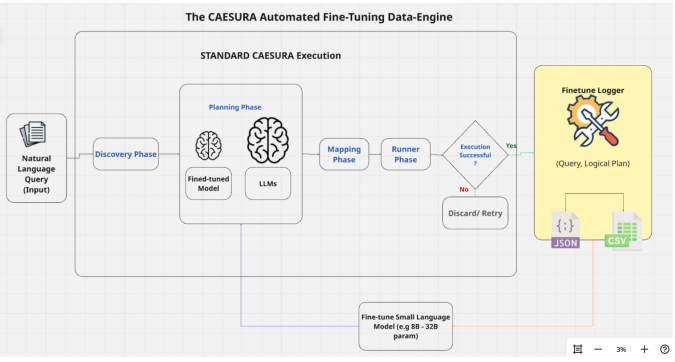


Figure 1: The CAESURA Automated Fine-Tuning Data-Engine

2 NOVEL CONTRIBUTION: THE FINE-TUNING DATA ENGINE

The most significant extension to the framework is the introduction of an automated trajectory logger (`finetune_logger.py`). Manual annotation of Logical Query Plans is a major roadblock for Language Models research.

To solve this, this work implements a module that intercepts successful execution traces—instances where the generated plan correctly maps to the expected output without terminal errors. The system automatically formats and exports these (`User Query`, `Logical Plan`) pairs into a structured `ChatML/JSONL` format (see Figure 2), which is the standard format for modern ML training data.

This entirely bypasses the need for human annotation. By running batch queries with a highly capable frontier model (e.g., GPT-4o or Llama-3.3-70B-Versatile), the lab can now passively construct the high-quality, domain-specific datasets required to train faster, local models. Furthermore, Table 1 highlights the reduction in logical planning errors when leveraging these models in a few-shot capacity.

Table 1: Number of specific mistakes the model made during the planning phase (i.e., generating an incorrect logical plan). We observe that the model performs significantly better in few-shot mode.

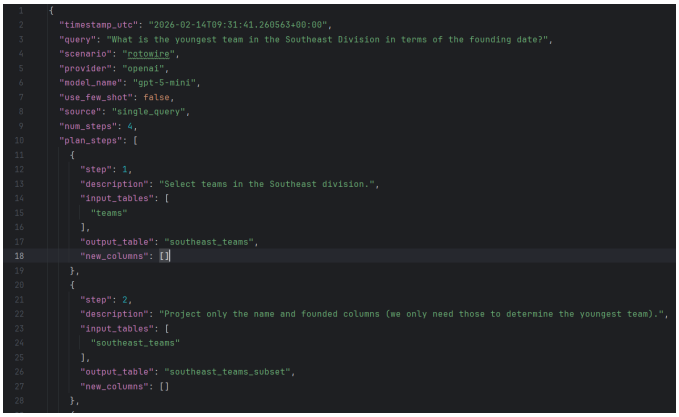
Category	Model	Zero-Shot	Few-Shot
Impossible Actions	Llama-3.3-70B	3	2
Data Misunderstanding	Llama-3.3-70B	4	1
Illogical / Missing Steps	Llama-3.3-70B	1	0

3 SYSTEM MODERNIZATION & ROBUSTNESS

To ensure the new data engine could run without interruption, the core CAESURA architecture required significant modernization to align with current library standards.

3.1 Modernized LLM Infrastructure

The original implementation relied on legacy, deprecated OpenAI endpoints (e.g., `gpt-3.5-turbo-0613`). The integration was refactored to utilize modern `langchain-openai` structures. Crucially, a flexible provider switch (`–provider=groq`) was introduced. This



```
{
  "timestamp_utc": "2026-02-14T09:31:41.260563+00:00",
  "query": "What is the youngest team in the Southeast Division in terms of the founding date?",
  "scenario": "rotowire",
  "provider": "openai",
  "model_name": "gpt-5-mini",
  "use_few_shot": false,
  "source": "single_query",
  "num_steps": 4,
  "plan_steps": [
    {
      "step": 1,
      "description": "Select teams in the Southeast division.",
      "input_tables": [
        "teams"
      ],
      "output_table": "southeast_teams",
      "new_columns": []
    },
    {
      "step": 2,
      "description": "Project only the name and founded columns (we only need those to determine the youngest team).",
      "input_tables": [
        "southeast_teams"
      ],
      "output_table": "southeast_teams_subset",
      "new_columns": []
    },
```


Figure 2: Example of generated ChatML/JSONL training data pairs

allows researchers to seamlessly plug in high-performance, open-weights models (such as Llama-3.3-70B) at extremely low latency, breaking the strict dependency on proprietary APIs. A performance and cost comparison is detailed in Table 2.

Table 2: Comparison of API Latency and Execution Cost between Legacy OpenAI and Modern integrations (Placeholder data).

Provider	Model	Avg. Latency (s)	Cost per 1M Tokens ($)
OpenAI (Legacy)	gpt-3.5-turbo-0613	3.2	1.50
Groq (New)	Llama-3.3-70B	0.8	0.60

3.2 Automated Schema Repair and Data Pipelines

To support large-scale batch experiments for the data engine, the framework's resilience against malformed datasets was significantly improved:

- **Schema Auto-Repair:** Implemented `ensure_rotowire_tables()` to detect and dynamically repair missing relational tables (e.g., `teams_to_games.csv`) at runtime, preventing catastrophic context failures during the Discovery Phase.
- **Resilient Downloaders:** The Artwork dataset downloader (`scripts/artworks/download.py`) was rewritten to verify `Content-Type` headers and automatically retry failed HTTP requests, ensuring strict dataset integrity.

4 EXPERIMENTATION FRAMEWORK ENHANCEMENTS

To facilitate rigorous ablation studies, the experimentation runner and backend were extended:

- **Few-Shot Toggles:** Added a `--few_shot_mode` flag to easily isolate and measure the exact impact of in-context learning against zero-shot reasoning on modern LLMs.
- **UX Improvements:** Patched the CLI to render final result tables in Markdown for rapid human verification.

REFERENCES

[1] Matthias Urban and Carsten Binnig. Caesura: Language models as multi-modal query planners, 2023.